

```
`timescale 1ns / 1ps
`default_nettype none
```

```
module tb_core;
```

```
    reg clk;
    reg rst_n;
```

```
    reg [31:0] i_i_data;
    reg [31:0] i_d_data;
    reg i_i_ack;
    reg i_d_ack;
```

```
    wire o_i_stb;
    wire o_d_stb;
    wire o_d_wr_en;
    wire [31:0] o_d_write_data;
    wire [31:0] o_i_addr;
    wire [31:0] o_d_addr;
```

```
    wire [31:0] if_instr, if_Pc;
    wire [31:0] id_rs1_data, id_rs2_data, id_imm_data, id_pc;
    wire [2:0] id_func3;
    wire [4:0] id_rd;
    wire id_regSrc, id_sel1, id_sel2, id_load, id_branch, id_jal, id_jalr,
id_memSrc;
```

```
    wire [1:0] id_resultSrc;
    wire [3:0] id_alu_ctrl;
    wire [31:0] ex_pc, ex_imm_data, ex_data_store;
    wire [2:0] ex_func3;
    wire [4:0] ex_rd;
    wire ex_load;
    wire [31:0] result_to_if;
    wire ex_boj, ex_jalr, ex_regsrc, ex_memsrc;
    wire [31:0] ex_result;
    wire [1:0] ex_resultsrc;
    wire [31:0] fwdwire1, fwdwire2;
    wire m_regSrc;
    wire [4:0] m_rd;
    wire [1:0] m_resultsrc;
    wire [31:0] m_result, m_data, m_pc4;
    wire w_regsrc;
    wire [4:0] w_rd;
    wire [31:0] w_data;
    wire o_load_use_hazard;
    wire [4:0] id_amo_op, ex_amo_op, m_amo_op;
```

```

core uut (
    .clk(clk),
    .rst_n(rst_n),
    .i_i_data(i_i_data),
    .i_d_data(i_d_data),
    .i_i_ack(i_i_ack),
    .i_d_ack(i_d_ack),
    .o_i_stb(o_i_stb),
    .o_d_stb(o_d_stb),
    .o_d_wr_en(o_d_wr_en),
    .o_d_write_data(o_d_write_data),
    .o_i_addr(o_i_addr),
    .o_d_addr(o_d_addr),
    .if_instr(if_instr),
    .if_Pc(if_Pc),
    .id_rs1_data(id_rs1_data),
    .id_rs2_data(id_rs2_data),
    .id_imm_data(id_imm_data),
    .id_pc(id_pc),
    .id_func3(id_func3),
    .id_rd(id_rd),
    .id_regSrc(id_regSrc),
    .id_sel1(id_sel1),
    .id_sel2(id_sel2),
    .id_load(id_load),
    .id_branch(id_branch),
    .id_jal(id_jal),
    .id_jalr(id_jalr),
    .id_memSrc(id_memSrc),
    .id_resultSrc(id_resultSrc),
    .id_alu_ctrl(id_alu_ctrl),
    .ex_pc(ex_pc),
    .ex_imm_data(ex_imm_data),
    .ex_data_store(ex_data_store),
    .ex_func3(ex_func3),
    .ex_rd(ex_rd),
    .ex_load(ex_load),
    .result_to_if(result_to_if),
    .ex_boj(ex_boj),
    .ex_jalr(ex_jalr),
    .ex_regsrc(ex_regsrc),
    .ex_memsrc(ex_memsrc),
    .ex_result(ex_result),
    .ex_resultsrc(ex_resultsrc),
    .fwdwire1(fwdwire1),

```

```

        .fwdwire2(fwdwire2),
        .m_regSrc(m_regSrc),
        .m_rd(m_rd),
        .m_resultsrc(m_resultsrc),
        .m_result(m_result),
        .m_data(m_data),
        .m_pc4(m_pc4),
        .w_regsrc(w_regsrc),
        .w_rd(w_rd),
        .w_data(w_data),
        .o_load_use_hazard(o_load_use_hazard),
        .id_amo_op(id_amo_op),
        .ex_amo_op(ex_amo_op),
        .m_amo_op(m_amo_op)
    );

// Clock generation
initial clk = 0;
always #5 clk = ~clk;

// Task to apply instruction with i_i_data and simulate ACK
task load_instr(input [31:0] instr);
    begin
        i_i_data = instr;
        i_i_ack = 1;
        #10;
        i_i_ack = 0;
    end
endtask

initial begin
$display("Starting Full AMO Testbench...");

// Reset
rst_n = 0;
i_i_data = 0;
i_d_data = 0;
i_i_ack = 0;
i_d_ack = 0;

#20;
rst_n = 1;

// Simulated memory content at address in x3
i_d_data = 32'h00000005;
i_d_ack = 1;

```

```

#20;

// Task call for each AMO instruction
// AMOSWAP.W
load_instr(32'h0803182f); // Swap x2 <-> MEM[x3]
#30 i_d_ack = 1;

// AMOADD.W
load_instr(32'h0003182f); // MEM[x3] += x2
#30 i_d_ack = 1;

// AMOAND.W
load_instr(32'h6003182f); // MEM[x3] &= x2
#30 i_d_ack = 1;

// AMOOR.W
load_instr(32'h4003182f); // MEM[x3] |= x2
#30 i_d_ack = 1;

// AMOXOR.W
load_instr(32'h2003182f); // MEM[x3] ^= x2
#30 i_d_ack = 1;

// Let pipeline settle
#100;

$display("Completed AMO instruction tests.");
$finish;

end
endmodule

```